CYA N

Choose Your AI - Noob

Documentazione Tecnica Architetturale

Versione 5.0 (Stabile)

22 febbraio 2026

Indice

1 Panoramica Architetturale e Vincoli di Sistema **1**
1.1 Definizione del Sistema . . . 1
1.2 Requisiti di Resilienza e Operatività Offline . . . 1
1.3 Vincoli Hardware e Modelli di Progettazione . . . 1

2 Topologia del Sistema e Grafi di Instradamento **3**
2.1 Mappatura delle Relazioni e Archi di Emergenza . . . 3
2.2 Struttura del Filesystem e Archi di Dipendenza . . . 3

3 Motore di Instradamento e Smistamento Intelligente V2.0 **5**
3.1 Analisi Lessicale e Tokenizzazione . . . 5
3.2 Complessità Algoritmica e Criteri di Valutazione . . . 5
3.3 Risoluzione dei Conflitti e Logiche di Priorità . . . 5
3.4 Limitazioni Attuali e Metodi di Tolleranza . . . 6

4 Centro di Controllo e Guardia Dinamica della RAM **7**
4.1 Centralizzazione Parametrica (`config.py`) . . . 7
4.2 Monitoraggio delle Risorse e Verifiche Hardware . . . 7
4.3 Algoritmo di Fallback Ricorsivo e Archi di Emergenza . . . 7

5 Core AI, Elaborazione Asincrona e Text Parsing **9**
5.1 Astrazione Logica e Polimorfismo (`BaseAI`) . . . 9
5.2 Iniezione del Contesto e Prompt Engineering . . . 9
5.3 Flusso di Streaming e Interfacciamento Server . . . 9
5.4 Sanificazione dell'Output e Filtri a Runtime . . . 10

6 Flotta LLM e Parametri di Inferenza **11**
6.1 Selezione e Quantizzazione dei Modelli . . . 11
6.2 Ingegneria delle Temperature . . . 11

7 Istruzioni Operative di Deploy e Installazione **13**
7.1 Inizializzazione del Server AI (Ollama) . . . 13
7.2 Provisioning della Flotta LLM . . . 13
7.3 Configurazione dell'Ambiente Python . . . 14
7.4 Avvio del Sistema . . . 14

8 Implementazioni Future e Ottimizzazioni **15**
8.1 Evoluzione del Motore di Instradamento . . . 15
8.2 Gestione dello Stato e Memoria di Conversazione . . . 15
8.3 Iniezione del Contesto Temporale . . . 15
8.4 Interfaccia Utente e Registri Linguistici . . . 15

Capitolo 1

Panoramica Architetturale e Vincoli di Sistema

1.1 Definizione del Sistema

CYA N (Choose Your AI - Noob) nasce con l'obiettivo chiaro di essere un **dispatcher** di richieste. Il software analizza l'istruzione ricevuta dall'utente e, tramite un controllo di corrispondenza basato sulle parole chiave (confrontate con i file dizionario presenti nella directory `keywords/`), traccia un arco di instradamento verso il modulo di competenza più adatto (Coding, Matematica, Diritto o Generale). A ogni modulo, infine, è associato tramite un arco di esecuzione un modello di AI dedicato, selezionato specificamente per le sue prestazioni ottimali in quel determinato dominio.

1.2 Requisiti di Resilienza e Operatività Offline

Un requisito funzionale primario di CYA N è la totale indipendenza dall'infrastruttura di rete. L'architettura è stata progettata per garantire un'operatività al 100% in locale. Di conseguenza, ogni scambio di dati e ogni arco di comunicazione tra il dispatcher, il motore logico e l'interfaccia utente si esaurisce strettamente all'interno della macchina fisica. Questo approccio elimina alla radice le latenze di rete e garantisce che i dati scambiati non vengano mai trasmessi all'esterno, offrendo un ambiente a privacy assoluta.

1.3 Vincoli Hardware e Modelli di Progettazione

Lo sviluppo del dispatcher è stato calibrato attorno a un limite hardware rigoroso: l'esecuzione fluida e stabile su macchine dotate di soli **8GB di RAM**. Per operare all'interno di questo perimetro fisico senza incorrere in crash di sistema, il codice implementa tre specifiche scelte architetturali:

- **Pattern Singleton:** Applicato alla classe di gestione dei file testuali (`KeywordLoader`). Assicura che i vocabolari di smistamento vengano allocati nella memoria di sistema una singola volta durante il ciclo di vita dell'applicativo.
- **Pattern Factory:** Utilizzato dal dispatcher per creare l'istanza dinamica dei modelli. Il sistema traccia un arco di creazione per costruire la classe AI appropriata esclusivamente a runtime, mantenendo il codice modulare e disaccoppiato.
- **Guardia Dinamica della RAM:** Un meccanismo di controllo preventivo (*pre-polling*). Prima di inizializzare il colloquio con un modello, il sistema interroga le

risorse fisiche della macchina: se la RAM residua è inferiore alla soglia richiesta, il software traccia immediatamente un arco di **fallback preventivo** verso un modello più leggero (modelli con un minor numero di parametri o maggiore quantizzazione), evitando le interruzioni per *Out of Memory* (OOM). Ciò garantisce che il modello rispondente possieda una quantità di memoria RAM sufficiente per fornire risposte precise in un tempo limitato.

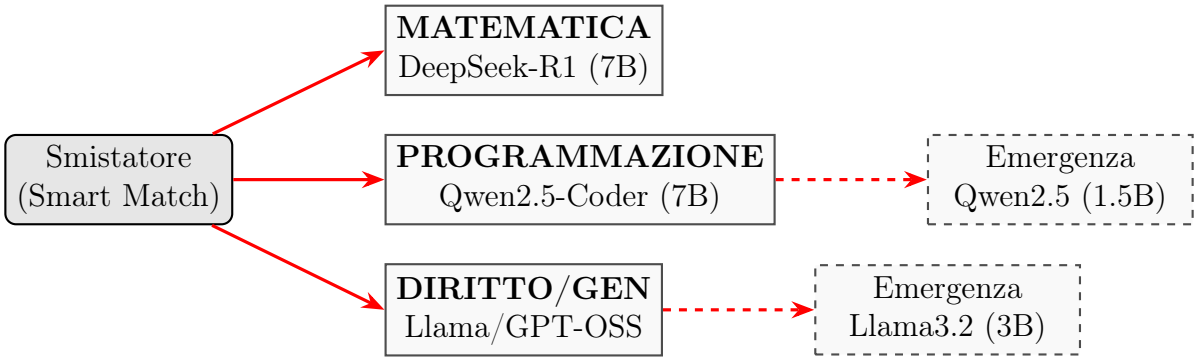
Capitolo 2

Topologia del Sistema e Grafi di Instradamento

2.1 Mappatura delle Relazioni e Archi di Emergenza

L'architettura modella il flusso di esecuzione come un grafo orientato, in cui ogni dipendenza logica o transizione di stato è formalizzata tramite archi. Il modulo di controllo delle risorse agisce come nodo centrale: valutando il costo in byte richiesto dal modello di destinazione, determina in tempo reale se confermare l'arco di esecuzione verso il modello principale o se deviare il carico computazionale.

Di seguito si riporta la topologia del routing. Per massimizzare la chiarezza visiva e strutturale, il grafo utilizza esclusivamente **frecce rosse** per tracciare gli archi che collegano i moduli, distinguendo i percorsi di instradamento standard (linea continua) dalle deviazioni di emergenza (linea tratteggiata):



2.2 Struttura del Filesystem e Archi di Dipendenza

L'organizzazione del filesystem separa rigorosamente la logica applicativa dai dati statici, stabilendo chiari archi di dipendenza tra i moduli di sistema per garantire alta coesione e basso accoppiamento.

- `code/`: Directory contenente il nucleo esecutivo dell'applicazione.

  - `main.py`: Punto di ingresso (CLI). Inizializza l'applicativo e traccia archi di dipendenza diretti verso lo smistatore e l'infrastruttura AI.
  - `config.py`: Centro di configurazione. Non possiede dipendenze in ingresso, ma espone archi di lettura globali verso l'intero sistema, fornendo parametri operativi e path di sistema.
  - `dispatcher_request.py`: Motore che esegue l'instradamento semantico basato sulle corrispondenze.
  - `ai_engine.py`: Astrazione logica deputata alla gestione dell'interfacciamento asincrono con il server locale.
  - `helper.py` e `prompts_templates.py`: Moduli di utilità per la sanificazione degli output testuali e l'iniezione dei contesti operativi.

- `keywords/`: Database testuale di routing, ospitante i vocabolari specializzati per dominio.

Capitolo 3

Motore di Instradamento e Smistamento Intelligente V2.0

3.1 Analisi Lessicale e Tokenizzazione

Il modulo `dispatcher_request.py` implementa la logica decisionale del sistema. L'elaborazione del linguaggio naturale (NLP) si basa su un algoritmo deterministico di estrazione e confronto (definito *Hard-Matching*).

Alla ricezione dell'input, il sistema avvia un **processo di normalizzazione** per convertire l'intero blocco di testo in caratteri minuscoli. Successivamente, tramite un set di espressioni regolari mirate (`re.findall(r'\w+', ...)`), scompone la stringa in token atomici, elidendo la punteggiatura e aggregando i termini in un insieme matematico (Set) privo di duplicati.

3.2 Complessità Algoritmica e Criteri di Valutazione

Per garantire latenze prossime allo zero nella fase di pre-elaborazione, il confronto tra i token in input e i dizionari di sistema avviene sfruttando le tabelle hash native delle strutture dati Set di Python, restituendo una complessità temporale asintotica pari a $\mathcal{O}(1)$.

Per ogni corrispondenza rilevata in fase di lookup, l'algoritmo registra una **correlazione**, incrementando il peso metrico assegnato al dominio di appartenenza (Programmazione, Matematica o Diritto). L'architettura esegue una doppia validazione, distinguendo l'analisi per token singoli da quella per n-grammi o stringhe composte.

3.3 Risoluzione dei Conflitti e Logiche di Priorità

In presenza di istruzioni eterogenee che generano molteplici correlazioni simultanee (es. un prompt che interseca logiche matematiche e codice sorgente), il sistema applica una logica di risoluzione deterministica basata su una **priorità assoluta**:

1. **Diritto:** Detiene la priorità massima di instradamento. Tale vincolo impedisce che terminologia generica possa invalidare il rigore legale richiesto in questo specifico dominio.

2. **Programmazione vs Matematica:** La risoluzione avviene per valutazione quantitativa diretta. Il dominio che registra il numero maggiore di correlazioni validate determina l'arco di instradamento definitivo. In caso di stallo semantico (pareggio

quantitativo), l'arco di esecuzione viene forzato verso la **Matematica**, al fine di preservare le capacità logico-deduttive del modello.

3. **Generale:** Agisce come nodo di fallback semantico, accogliendo l'esecuzione esclusivamente in assenza di correlazioni specializzate.

3.4 Limitazioni Attuali e Metodi di Tolleranza

L'attuale approccio basato sull'*Hard-Matching* offre prestazioni computazionali ottimali ma introduce limitazioni di flessibilità operativa. Poiché una correlazione si verifica esclusivamente a fronte di un'uguaglianza rigorosa (`token == keyword`), l'algoritmo rigetta variazioni morfologiche, plurali o errori ortografici.

L'evoluzione architetturale pianificata per questo modulo prevede l'integrazione del calcolo della **Distanza di Levenshtein**. Questo aggiornamento sostituirà i vincoli di uguaglianza binaria con **metodi di tolleranza elastici**, garantendo flessibilità e precisione nell'instradamento, pur mantenendo il carico computazionale confinato nei limiti hardware imposti dal sistema.

Capitolo 4

Centro di Controllo e Guardia Dinamica della RAM

4.1 Centralizzazione Parametrica (`config.py`)

Il modulo `config.py` agisce come singola fonte di verità per l'intero ecosistema CYA N. Isolando le costanti hardware, i limiti di memoria e i percorsi assoluti dal codice esecutivo, l'architettura garantisce un'elevata scalabilità e manutenibilità. Questo file istituisce un arco di dipendenza passiva: gli altri moduli interrogano le sue strutture dati (come `MODELS_CONFIG` e `RAM_THRESHOLDS`) a runtime, prelevando i parametri di esecuzione senza alterare la logica del sistema.

4.2 Monitoraggio delle Risorse e Verifiche Hardware

Operare all'interno di un vincolo fisico di 8GB di memoria RAM richiede una gestione difensiva delle risorse. CYA N supera questo limite implementando un meccanismo di controllo preventivo basato sull'interfacciamento hardware tramite la libreria `psutil`.

Prima di avviare l'elaborazione dei tensori e interrogare il server locale, il modulo `BaseAI` effettua un arco di verifica hardware. Questo processo interroga direttamente il sistema operativo per calcolare la memoria virtuale istantaneamente disponibile, restituendo un valore in byte che funge da parametro decisionale per il caricamento del modello.

4.3 Algoritmo di Fallback Ricorsivo e Archi di Emergenza

La logica decisionale per l'allocazione delle risorse si affida a un algoritmo di controllo a cascata, il quale definisce rigorosi archi di esecuzione condizionali:

1. **Valutazione Primaria:** Il sistema confronta la memoria disponibile con la soglia limite definita per il modello primario (es. 5.5GB per i modelli a 7B parametri). Se la validazione ha esito positivo, si avvia l'arco di esecuzione standard verso l'agente principale.

2. **Downgrade Preventivo:** Qualora la verifica hardware rilevi una disponibilità RAM inferiore alla soglia critica, il sistema intercetta la potenziale eccezione di *Out of Memory* (OOM) in via del tutto preventiva. Viene immediatamente tracciato

un arco di emergenza (fallback) che reindirizza la chiamata verso un modello più leggero e quantizzato (es. 1.5B o 3B parametri).

3. **Arresto di Sicurezza:** Se il sistema rileva che le risorse risultano insufficienti persino per instanziare il modello di emergenza, l'orchestratore inibisce l'operazione. Questo arco di interruzione volontaria previene il blocco del sistema operativo host, restituendo un avviso formale all'utente.

Tale meccanismo rende l'infrastruttura altamente resiliente. Assicura che un arco di risposta con l'utente venga mantenuto sempre attivo, sacrificando marginalmente la complessità del ragionamento logico a favore della stabilità e della velocità di calcolo complessiva.

Capitolo 5

Core AI, Elaborazione Asincrona e Text Parsing

5.1 Astrazione Logica e Polimorfismo (`BaseAI`)

L'interfacciamento con i Large Language Models (LLM) è gestito tramite un'architettura orientata agli oggetti basata su classi astratte. Il modulo `ai_engine.py` definisce la classe base `BaseAI`, la quale incapsula la logica comune di controllo delle risorse e di generazione del testo. Da questa classe genitrice si diramano molteplici archi di ereditarietà verso le implementazioni specifiche dei domini (`CodeLlamaAI`, `DeepSeekAI`, `GptOssAI`).

La selezione del modello operativo avviene tramite il pattern Factory (`get_ai_model`): a runtime, in base alla categoria individuata dal dispatcher, viene tracciato un arco di istanziazione che inizializza la classe derivata corretta. Questo approccio garantisce un elevato disaccoppiamento dei componenti e facilita l'integrazione futura di nuovi modelli.

5.2 Iniezione del Contesto e Prompt Engineering

Prima di avviare l'inferenza, il sistema struttura il payload informativo interrogando il modulo `prompts_templates.py`. Questo file agisce come repository centralizzato per i *System Prompts* (le istruzioni di base che definiscono il ruolo dell'agente) e per gli esempi *Few-Shot*.

L'assemblaggio del prompt finale stabilisce un arco di condizionamento semantico che vincola lo spazio latente del modello. Ad esempio, per il dominio matematico, vengono iniettati parametri di *enforcement* che impongono la risoluzione passo-passo e l'esclusivo utilizzo della lingua italiana, bypassando le tendenze linguistiche native del modello DeepSeek.

5.3 Flusso di Streaming e Interfacciamento Server

La generazione della risposta non avviene in blocco, ma attraverso un arco di comunicazione asincrona con il server locale. Il metodo `generate()` instaura una connessione in streaming, permettendo all'applicazione di ricevere e processare i frammenti di testo non appena vengono elaborati dalla GPU/CPU.

Questa architettura riduce drasticamente il tempo di primo byte (TTFB - *Time To First Byte*) percepito dall'utente. Durante l'attesa iniziale, il processo principale genera un thread separato che gestisce un feedback visivo (spinner), mantenendo l'interfaccia a riga di comando fluida e reattiva.

5.4 Sanificazione dell'Output e Filtri a Runtime

I flussi di testo grezzi provenienti dai modelli, in particolare quelli dotati di architetture di *Reasoning* (come DeepSeek-R1), contengono metadati interni e formattazioni incompatibili con la console di sistema.

Il modulo `helper.py` intercetta il flusso in uscita applicando un rigoroso arco di sanificazione testuale. La funzione `clean_response()` esegue tre operazioni sequenziali con complessità temporale $\mathcal{O}(N)$:

1. **Rimozione Chain of Thought:** Filtraggio tramite espressioni regolari (`re.sub`) dei blocchi di ragionamento latente racchiusi nei tag `<think>`.

2. **Normalizzazione Alfabetica:** Eliminazione di eventuali caratteri CJK (cinese, giapponese, coreano) spesso generati da allucinazioni nei modelli fortemente quantizzati.

3. **Parsing LaTeX:** Sostituzione a runtime dei costrutti matematici complessi (es. `\frac`, `\sum`) con i corrispondenti caratteri Unicode, al fine di garantire una renderizzazione pulita e leggibile sul terminale standard dell'utente.

Capitolo 6

Flotta LLM e Parametri di Inferenza

6.1 Selezione e Quantizzazione dei Modelli

L'architettura di CYA N demanda l'esecuzione logica a una flotta di Large Language Models (LLM) eseguiti in locale. Per rispettare il vincolo hardware degli 8GB di RAM, tutti i modelli selezionati operano con pesi neurali fortemente quantizzati. Questa tecnica di compressione algoritmica riduce drasticamente l'impronta in memoria, preservando al contempo gran parte delle capacità inferenziali originarie.

La configurazione dei modelli (`MODELS_CONFIG`) è strutturata in base ai domini di competenza:

- **Programmazione (Coding):** Utilizza `qwen2.5-coder:7b` come nodo primario. Questo modello è stato addestrato specificamente su dataset di codice sorgente, offrendo prestazioni ottimali nel refactoring e nel debugging. Il modello di emergenza associato è `qwen2.5-coder:1.5b`, che garantisce la continuità operativa in condizioni di memoria critica (soglia < 5.5GB).

- **Matematica (Math):** Basato sull'architettura `deepseek-r1:7b`. L'assenza di un modello di emergenza per questo modulo è una scelta architetturale deliberata: la risoluzione matematica richiede capacità di *Reasoning* (Catena di Pensiero strutturata) che i modelli con un numero di parametri inferiore a 7 miliardi non possiedono in misura affidabile.

- **Diritto e Generale (Rights/General):** Il dominio legale e generalista sfrutta `gpt-oss:20b` per le interrogazioni complesse e devia su `llama3.2:3b` qualora le risorse di sistema risultino insufficienti per allocare modelli di grandi dimensioni.

6.2 Ingegneria delle Temperature

La generazione stocastica del testo è governata dal parametro di *Temperatura*, che modula la distribuzione di probabilità dei token in output. I valori sono stati tarati staticamente nel file di configurazione per massimizzare la coerenza logica di ciascun dominio:

- **Math ($T = 0.2$):** Valore prossimo allo zero per forzare un output deterministico, requisito essenziale per garantire la precisione nei calcoli numerici e nelle dimostrazioni.

- **Rights ($T = 0.4$):** Configurazione conservativa per prevenire allucinazioni in ambito legale, impedendo rigorosamente la generazione di riferimenti normativi fittizi o articoli inesistenti.

- **Coding ($T = 0.5$):** Bilanciamento calcolato tra sintassi deterministica e la flessibilità necessaria per proporre soluzioni algoritmiche alternative e refactoring eleganti.
- **General ($T = 0.7$):** Valore standard per favorire un linguaggio discorsivo, fluido e naturale nelle conversazioni non tecniche.

Capitolo 7

Istruzioni Operative di Deploy e Installazione

7.1 Inizializzazione del Server AI (Ollama)

Il primo passo per garantire la distribuzione e il funzionamento del progetto in qualsiasi ambiente consiste nell'installazione del motore di inferenza. Per stabilire l'**arco di hosting** locale, è necessario scaricare e installare Ollama.

1. Visitare il sito ufficiale: https://ollama.com.
2. Scaricare la versione compatibile con il proprio sistema operativo (Windows, macOS o Linux).
3. Eseguire l'installer seguendo le istruzioni a schermo e assicurarsi che il servizio sia attivo in background sul proprio sistema.

7.2 Provisioning della Flotta LLM

Una volta instaurato l'arco di comunicazione di base con il demone di Ollama, è obbligatorio scaricare localmente i pesi neurali dei modelli previsti dall'architettura di CYA N. Aprire il terminale di sistema e digitare i seguenti comandi per tracciare gli **archi di approvvigionamento** (*pulling*):

- **Dominio Programmazione (Primario):**
  `ollama pull qwen2.5-coder:7b`
- **Dominio Programmazione (Emergenza):**
  `ollama pull qwen2.5-coder:1.5b`
- **Dominio Matematica:**
  `ollama pull deepseek-r1:7b`
- **Dominio Diritto/Generale (Primario):**
  `ollama pull gpt-oss:20b`
- **Dominio Diritto/Generale (Emergenza):**
  `ollama pull llama3.2:3b`

Nota: L'operazione di download può richiedere diverso tempo a seconda della larghezza di banda disponibile. Il fallimento di un singolo pull interromperà i futuri fallback preventivi del sistema.

7.3 Configurazione dell'Ambiente Python

Il software richiede un ambiente Python 3.8 o superiore per poter eseguire il dispatcher. Per definire in modo rigoroso tutti gli **archi di dipendenza** software necessari all'esecuzione, è richiesto l'utilizzo del gestore pacchetti `pip`.

Da terminale, assicurandosi di avere i privilegi necessari, eseguire l'installazione delle librerie:

```
pip install psutil
```

Il modulo `psutil` è un requisito stringente, in quanto indispensabile per permettere al sistema di interrogare il sistema operativo e generare l'arco di verifica hardware essenziale per monitorare i blocchi di Out of Memory (OOM).

7.4 Avvio del Sistema

Una volta completato il setup del server, scaricati tutti i modelli e risolte le dipendenze Python, l'infrastruttura è pronta all'uso.

Per inizializzare l'orchestrazione e avviare il primo **arco di interazione** con l'utente, posizionarsi con il terminale nella cartella radice del progetto ed eseguire il comando:

```
python code/main.py
```

Capitolo 8

Implementazioni Future e Ottimizzazioni

8.1 Evoluzione del Motore di Instradamento

L'attuale sistema di smistamento si è dimostrato computazionalmente efficiente ma vulnerabile agli errori di digitazione, poiché la variazione di un singolo carattere impedisce il riconoscimento della parola chiave da parte dell'algoritmo. Per ovviare a questa rigidità strutturale, la successiva iterazione architetturale implementerà il calcolo della Distanza di Levenshtein. Tale ottimizzazione introdurrà un margine di tolleranza quantificabile agli errori ortografici, garantendo un instradamento corretto anche in presenza di input testuali imperfetti.

8.2 Gestione dello Stato e Memoria di Conversazione

Nella Versione 5.0, CYA N opera in modalità strettamente *stateless*: ogni elaborazione è indipendente e il sistema non conserva il contesto delle interazioni precedenti. Il piano di sviluppo prevede l'integrazione di un modulo di gestione della *Chat History*. Questo componente serializzerà i log di sessione in una struttura dati dedicata, iniettandoli iterativamente all'interno dei prompt per permettere ai modelli di mantenere coerenza logica nei dialoghi multi-turno.

8.3 Iniezione del Contesto Temporale

Per massimizzare l'affidabilità delle inferenze, l'architettura di iniezione del contesto verrà estesa per dotare le intelligenze artificiali della consapevolezza del momento storico corrente. A runtime, l'applicativo estrarrà la data e l'ora dal sistema operativo host, allegandole dinamicamente ai System Prompts prima di ogni chiamata al server, ancorando così il modello alle coordinate temporali reali.

8.4 Interfaccia Utente e Registri Linguistici

L'esperienza interattiva sarà potenziata evolvendo l'attuale input standard in un'interfaccia a riga di comando (CLI) *user friendly*, provvista di routine grafiche e animazioni di caricamento. Parallelamente, verrà implementato un selettore parametrico per il registro linguistico: l'operatore potrà commutare la sintassi dell'agente da un tono formale e accademico ("Professore") a un registro informale ("Amico"), senza alterare la logica di

calcolo sottostante. Infine, il sistema integrerà un manuale di sistema richiamabile tramite linea di comando per istruire l'utente sull'utilizzo ottimale dei prompt.

Capitolo 9

Glossario Tecnico

Dispatcher (Instradatore): Componente architetturale deputato all'analisi di un input testuale. Sulla base delle concordanze rilevate, traccia un arco di instradamento vincolante verso il nodo o il modulo di calcolo più appropriato.

Enforcement Prompt: Regola restrittiva accodata dinamicamente alla richiesta dell'utente. Genera un arco di costrizione logica che forza l'agente a ignorare le proprie tendenze native (es. rispondere in lingua inglese) a favore di output predeterminati.

Fallback Preventivo: Meccanismo di tolleranza ai guasti (Fault Tolerance) che intercetta a runtime le condizioni critiche del sistema (es. memoria RAM insufficiente) e devia l'arco di esecuzione verso una soluzione di riserva meno dispendiosa, prevenendo eccezioni bloccanti.

Few-Shot Prompting: Tecnica di iniezione del contesto in cui vengono forniti esempi pre-compilati al modello. Instanzia un arco di emulazione diretto tra il template di riferimento fornito e la struttura della risposta attesa.

Hard-Matching: Metodologia di validazione lessicale che richiede un'esatta e rigorosa corrispondenza, carattere per carattere, tra due stringhe di testo per poter instaurare un arco di correlazione semantica.

LLM (Large Language Model): Modello di intelligenza artificiale basato su reti neurali profonde, pre-addestrato su vastissime moli di dati testuali per comprendere, elaborare e generare linguaggio naturale.

Out of Memory (OOM): Stato di errore fatale in cui un processo esaurisce la memoria RAM fisica e virtuale allocabile dal sistema operativo, causando la rottura irreparabile di ogni arco di elaborazione in corso.

Quantizzazione: Algoritmo di compressione matematica che riduce la precisione numerica dei parametri di un modello neurale (ad esempio passando da Float16 a Int4), diminuendo drasticamente l'occupazione in memoria a fronte di una minima riduzione dell'accuratezza.

Reasoning (Catena di Pensiero): Capacità di specifici modelli AI di frammentare un problema analitico o matematico in sequenze logiche intermedie (*Chain of Thought*), tracciando archi deduttivi espliciti prima di restituire l'output finale.

Stateless (Senza stato): Architettura di elaborazione in cui ogni transazione viene trattata come un processo isolato, privo di archi temporali o di memoria condivisa che lo colleghino alle esecuzioni precedenti.

Streaming (Asincrono): Processo di ricezione progressiva dell'output in frammenti (chunk). Apre un arco di comunicazione continuo in tempo reale tra il server AI locale e l'interfaccia utente, abbattendo la latenza percepita.

System Prompt: Direttiva inibitoria o di configurazione inviata a un LLM in modo invisibile all'utente. Stabilisce l'arco comportamentale di base definendo i vincoli operativi, la formattazione e il ruolo dell'agente.

Temperatura: Parametro inferenziale stocastico che regola la casualità nella generazione dei token. Un valore prossimo allo zero forza un arco di risposta deterministico e conservativo, mentre un valore alto espande la varianza probabilistica delle soluzioni proposte.

Tokenizzazione: Procedura di conversione che scompone una stringa di testo in unità atomiche (token), permettendo a un algoritmo di costruire archi di indicizzazione per valutare i dati lessicali.